

Coding_Standards

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
 - 1.1 Purpose
 - 1.2 Scope
2. Java Coding Standards
 - 2.1 Naming Conventions
 - 2.2 Code Format
 - 2.3 Programming Rules
 - 2.4 Best Practices
3. Python Coding Standards
 - 3.1 Naming Conventions
 - 3.2 Code Format
 - 3.3 Programming Rules
 - 3.4 Best Practices
4. Go Coding Standards
 - 4.1 Naming Conventions
 - 4.2 Programming Rules
 - 4.3 Best Practices
5. Spring Boot Standards
 - 5.1 Project Structure
 - 5.2 Layer Rules
 - 5.3 Configuration Management
6. React Standards
 - 6.1 Project Structure
 - 6.2 Component Rules
 - 6.3 Code Example
7. Database Object Naming Conventions
 - 7.1 Naming Rules
 - 7.2 Table Name Rules
 - 7.3 Column Rules
 - 7.4 SQL Standards

Coding Standards

> Reference: Alibaba Java Development Manual, Google Style Guides, Tencent Coding Standards, Uber Go Style Guide

1. Overview

1.1 Purpose

Standardize coding styles across languages and frameworks within the Technology Development Department, improving code readability, maintainability, and reviewability, and reducing team collaboration friction.

1.2 Scope

This standard covers Java, Python, and Go programming languages, Spring Boot and React frameworks, and database object naming conventions.

2. Java Coding Standards

2.1 Naming Conventions

Element	Convention	Example
Class/Interface	UpperCamelCase	PolicyServiceImpl
Method	lowerCamelCase	getPolicyByNo()
Constant	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
Variable	lowerCamelCase	policyNo
Package	all lowercase with .	com.company.insurance.underwriting
Abstract Class	Abstract prefix	AbstractPolicyValidator
Exception Class	Exception suffix	PolicyNotFoundException
Test Class	Test suffix	PolicyServiceTest

2.2 Code Format

- Indentation: 4 spaces (no Tabs)
- Line width: max 120 characters
- Braces: K&R style (opening brace on same line)
- Blank lines: separate class members, 1 blank line between methods

2.3 Programming Rules

Exception Handling:

```
// Preferred: use business exceptions over return codes
if (policy == null) {
    throw new PolicyNotFoundException("Policy not found: " + policyNo);
}
// Forbidden: catch Exception without handling
try {
    // ...
} catch (Exception e) {
    // At minimum, log the error
    log.error("Processing failed", e);
}
```

Collections:

```
// Preferred: explicit generic types
List<String> policyNoList = new ArrayList<>();
// Preferred: use Stream API
List<String> activePolicies = policyList.stream()
    .filter(p -> "ACTIVE".equals(p.getStatus()))
    .map(Policy::getPolicyNo)
    .collect(Collectors.toList());
```

Concurrency:

- Use `ThreadPoolExecutor` with explicit parameters instead of `Executors.newFixedThreadPool()`
- Use `CompletableFuture` over manual `FutureTask`
- Accounting operations must use distributed locks (Redis/etcd), never local locks

2.4 Best Practices

- Use Lombok to reduce boilerplate code
- Use MapStruct for object mapping
- Use Builder pattern for complex object construction
- Use SLF4J facade + Logback for logging

3. Python Coding Standards

3.1 Naming Conventions

Element	Convention	Example
Module	lowercase_with_underscores	<code>policy_service.py</code>
Class	UpperCamelCase	<code>PolicyService</code>
Function/Method	lowercase_with_underscores	<code>get_policy_by_no()</code>
Variable	lowercase_with_underscores	<code>policy_no</code>
Constant	UPPER_SNAKE_CASE	<code>MAX_RETRY_COUNT</code>
Private Method	single underscore prefix	<code>_validate_policy()</code>

3.2 Code Format

- Indentation: 4 spaces
- Line width: max 88 characters (Black default)
- Must use Black + isort for formatting
- Type annotations are mandatory

3.3 Programming Rules

```
from typing import Optional
from decimal import Decimal
class PolicyService:
    """Policy service"""
    def get_policy(self, policy_no: str) -> Optional[Policy]:
        """Query policy by policy number"""
        pass
    def calculate_premium(
        self,
        base_amount: Decimal,
        loading_rate: Decimal,
    ) -> Decimal:
        """Calculate premium using Decimal for precision"""
        return base_amount * (Decimal("1") + loading_rate)
```

3.4 Best Practices

- Use Pydantic for data validation and serialization
- Use SQLAlchemy 2.0+ (declarative mapping)
- Use `match...case` instead of excessive `if/elif` (Python 3.10+)
- Prefer `asyncio` + `async/await` for async code

4. Go Coding Standards

4.1 Naming Conventions

Element	Convention	Example
Package	all lowercase, singular	policy, underwriting
Exported type	PascalCase	PolicyService
Unexported type	camelCase	policyRepo
Interface	er suffix	PolicyRepository
Error variable	err / Err prefix	ErrPolicyNotFound

4.2 Programming Rules

```
package policy
import "errors"
var ErrPolicyNotFound = errors.New("policy not found")
// PolicyService handles policy business logic
type PolicyService struct {
    repo PolicyRepository
}
// GetPolicy retrieves a policy by its number
func (s *PolicyService) GetPolicy(ctx context.Context, policyNo string) (*Policy, error) {
    if policyNo == "" {
        return nil, errors.New("policy number must not be empty")
    }
    return s.repo.FindByNo(ctx, policyNo)
}
```

4.3 Best Practices

- Always check error return values; never ignore with `_`
- Use `errors.Is()` / `errors.As()` for error evaluation
- Use `sync/atomic` over `sync.Mutex` for simple counters
- Use `Wire` / `fx` for dependency injection

5. Spring Boot Standards

5.1 Project Structure

```
src/main/java/com/company/module/
├── controller/ # REST controllers
├── service/ # Business logic
├── repository/ # Data access
├── model/entity/ # Data entities
├── model/dto/ # Data transfer objects
├── model/vo/ # View objects
├── config/ # Configuration classes
├── handler/ # Global exception handling
├── interceptor/ # Interceptors
└── util/ # Utility classes
```

5.2 Layer Rules

Layer	Responsibility	Prohibited
Controller	Parameter validation, routing	Must not contain business logic
Service	Business orchestration, transaction management	Must not directly access database
Repository	Data access	Must not contain cross-table business logic

- `@Transactional` must be declared on Service layer methods
- Controllers must not return Entity objects directly; use DTOs
- Parameter validation via `@Valid` / `@Validated`

5.3 Configuration Management

- Environment config: `application-{profile}.yaml` (dev/test/staging/prod)
- Secrets via configuration center (Nacos/Apollo)
- Hardcoding configuration in code is prohibited

6. React Standards

6.1 Project Structure

```
src/
├── components/ # Shared components
├── pages/ # Page components
├── hooks/ # Custom Hooks
├── services/ # API calls
├── stores/ # State management
├── types/ # TypeScript type definitions
├── utils/ # Utility functions
└── constants/ # Constants
```

6.2 Component Rules

- Use Function Components + Hooks, not Class Components
- Use TypeScript; any types are forbidden
- Single responsibility per component, max 200 lines
- Use `React.memo` for frequently re-rendered components
- State management: prefer React Context / Zustand; use Redux Toolkit only as needed

6.3 Code Example

```
import React, { useState, useCallback } from 'react';
import { Policy } from '@types/policy';
interface PolicySearchProps {
  onSearch: (keyword: string) => Promise<Policy[]>;
}
export const PolicySearch: React.FC<PolicySearchProps> = ({ onSearch }) => {
  const [keyword, setKeyword] = useState('');
  const [loading, setLoading] = useState(false);
  const handleSearch = useCallback(async () => {
    setLoading(true);
    try {
      await onSearch(keyword);
    } finally {
      setLoading(false);
    }
  }, [keyword, onSearch]);
  return ( /* JSX */ );
};
```

```
};
```

7. Database Object Naming Conventions

7.1 Naming Rules

Object	Convention	Example
Table	lowercase + underscore	t_policy_info
Column	lowercase + underscore	policy_no
Primary Key	pk_{table_name}	pk_t_policy_info
Index	idx_{table}_{column}	idx_t_policy_info_policy_no
Unique Constraint	uk_{table}_{column}	uk_t_policy_info_contract_no
Foreign Key	fk_{table}_{ref_table}	fk_t_policy_t_product

7.2 Table Name Rules

- Business tables: t_ prefix; dictionary/config tables: d_ prefix; log tables: l_ prefix
- Use singular nouns: t_policy not t_policies
- Relationship tables: _rel suffix: t_user_role_rel

7.3 Column Rules

- All tables must include: id (auto-increment/Snowflake), create_time, update_time, deleted (soft delete)
- Monetary fields: DECIMAL(18,2), never FLOAT/DOUBLE
- Date fields: DATETIME(3) (millisecond precision)
- Status fields: TINYINT with comment describing enum values

7.4 SQL Standards

```
-- Keywords uppercase, table/column names lowercase
SELECT
p.policy_no,
p.premium
FROM
t_policy_info p
WHERE
p.status = 1
AND p.create_time >= '2024-01-01'
ORDER BY
p.create_time DESC;
```

- SELECT * is prohibited
- Complex queries must use EXPLAIN for execution plan analysis
- JOIN operations on large tables must evaluate index usage

8. Appendix

8.1 Reference Documents

- *Alibaba Java Development Manual*
- Google Java Style Guide
- Python PEP 8 / PEP 484
- Uber Go Style Guide

- *Spring Boot Official Documentation*
- React TypeScript Cheatsheet

8.2 Tool Configuration

- Checkstyle / PMD (Java)
- Black + isort + flake8 + mypy (Python)
- golangci-lint (Go)
- ESLint + Prettier + TypeScript (React)
- SchemaSpy (database model visualization)